

Patrones de scripts que veo

Este es el orden más o menos que siguen la mayoría de ejercicios o las distintas opciones que ofrecen los ejercicios.

#	Patrón
1	Validación de argumentos (<code>\$#</code> , <code>\$1</code>)
2	Entrada interactiva (<code>read</code>)
3	Operaciones ficheros/directorios
4	Menú interactivo (<code>while</code> + <code>case</code>)
5	Bucles (<code>for</code> , <code>while</code> , <code>until</code>)
6	Funciones
7	Procesar <code>/etc/passwd</code> y <code>/etc/shadow</code>
8	Gestión de usuarios (root)
9	Manipulación de cadenas
10	Redes e IPs
11	Redirecciones y pipes
12	Comandos del sistema

Vamos a ver paso a paso cómo se realiza cada uno de estos pasos.

Patrón 1: Validación de Argumentos

Esto va a estar en prácticamente todos los ejercicios. Cómo lo podemos hacer: de la siguiente forma.

1a. Comprobar número exacto de argumentos

```
if [[ $# -ne 2 ]]; then
    echo "Uso: $0 <arg1> <arg2>"
    exit 1
fi
```

1b. Comprobar mínimo de argumentos

```
if [[ $# -lt 2 ]]; then
    echo "Uso: $0 <directorio> <palabra>"
    exit 1
fi
```

1c. Comprobar que NO se pasa argumento vacío

```
if [[ -z "$1" ]]; then
    echo "Uso: $0 <archivo>"
    exit 1
fi
```

1d. Validar opciones con `case`

```
case "$1" in
    -c) crear_copia "$2" ;;
    -r) restaurar_copia "$2" ;;
    *) echo "Opción no válida"; exit 1 ;;
esac
```

1e. Parseo con `getopts`

```
while getopts "pua" opcion; do
    case $opcion in
        p) ejecutar_permisos=true ;;
        u) ejecutar_usuarios=true ;;
        a) ejecutar_permisos=true; ejecutar_usuarios=true ;;
        *) echo "Error"; exit 1 ;;
    esac
done
```

1f. Parseo manual de flags y argumentos

```
for arg in "$@"; do
    if [[ "$arg" = "-f" ]]; then
        MOSTRAR_FICHEROS=1
    else
        DIRECTORIO="$arg"
    fi
done
```

1g. Obtener el último argumento

```
ruta_objetivo="${!#}"
```

1h. Argumento con valor por defecto

```
DIR="${1:-.}" # Si no se pasa $1, usa el directorio actual
```

Patrón 2: Entrada Interactiva (`read`)

A veces no se pide exactamente argumentos, en su lugar se van pidiendo datos y se van recogiendo mediante un prompt por consola. Para ello lo importante es el "read -p", que a veces se modifica con un "-s" si no se quiere mostrar.

2a. Lectura básica

```
read -p "Introduce un dato: " variable
```

2b. Lectura oculta (contraseñas)

```
read -r -s -p "Contraseña: " password  
echo # Salto de línea tras entrada oculta
```

2c. Lectura de múltiples valores

```
read -p "Introduce 2 números: " num1 num2
```

2d. Validar que no esté vacío

```
if [[ -z "$variable" ]]; then  
    echo "Error: No has introducido nada."  
    exit 1  
fi
```

2e. Validar que sea un número

```
if [[ ! "$num" =~ ^[0-9]+$ ]]; then  
    echo "Error: Introduce un número válido."  
    exit 1  
fi
```

2f. Insistir hasta dato válido (con límite)

```
intentos=0
while [[ -z "$usuario" ]] && (( intentos < 3 )); do
    read -r -p "Usuario: " usuario
    if [[ -z "$usuario" ]]; then
        echo "No puede estar vacío. Intento $((intentos+1)) de 3."
        (( intentos++ ))
    fi
done
```

2g. Confirmación de contraseña

```
read -r -s -p "Contraseña: " pass1; echo
read -r -s -p "Repita: " pass2; echo
if [[ "$pass1" != "$pass2" ]]; then
    echo "No coinciden."
    exit 1
fi
```

Patrón 3: Operaciones con Ficheros y Directorios

Comprobar si existe un directorio, un fichero, si tiene permiso, etc...

3a. Tests de ficheros (la tabla clave)

Test	Significado
-f "\$f"	Es fichero regular
-d "\$d"	Es directorio
-e "\$e"	Existe (fichero o dir)
-s "\$f"	Existe y tamaño > 0
-r "\$f"	Tiene permiso lectura
-w "\$f"	Tiene permiso escritura
-x "\$f"	Tiene permiso ejecución
-z "\$v"	Cadena vacía
-n "\$v"	Cadena NO vacía
-L "\$f"	Es enlace simbólico
-b "\$f"	Dispositivo de bloques
-c "\$f"	Dispositivo de caracteres
-p "\$f"	Es un FIFO (named pipe)

3b. Plantilla: verificar existencia

```
# Verificar fichero
if [[ ! -f "$1" ]]; then
    echo "El archivo $1 no existe."
    exit 1
fi

# Verificar directorio
if [[ ! -d "$DIR" ]]; then
    echo "El directorio $DIR no existe."
    exit 1
fi
```

3c. Iterar sobre ficheros de un directorio

```
for file in "$DIR"/*; do
    if [[ -f "$file" ]]; then
        echo "Archivo: $(basename "$file")"
    fi
done
```

3d. Crear backup de un fichero

```
cp "$1" "$1.bak"
```

3e. Crear directorio si no existe

```
mkdir -p "$directorio"
```

3f. Comprimir directorio con `tar`

```
tar -czf archivo.tar.gz directorio/
```

3g. Buscar archivos con `find`

```
# Buscar .txt en un directorio (sin recursión)
find "$DIR" -maxdepth 1 -type f -name "*.txt" | wc -l

# Buscar archivos grandes
find "$DIR" -type f -size +10M -exec ls -lh {} \;

# Buscar archivos de un usuario
find / -user "$usuario" -type f 2>/dev/null | wc -l
```

3h. Cambiar extensiones masivamente

```
shopt -s nullglob
for file in "$DIR"/*.txt; do
    mv "$file" "${file%.txt}.bak"
done
```

3i. Buscar texto en archivos

```
grep -rwl "palabra" "$DIR"
```

3j. Verificar permisos

```
[ -r "$FILE" ] && echo "Lectura: Sí" || echo "Lectura: NO"
[ -w "$FILE" ] && echo "Escritura: Sí" || echo "Escritura: NO"
[ -x "$FILE" ] && echo "Ejecución: Sí" || echo "Ejecución: NO"
```

Patrón 4: Menú Interactivo

Esto suele ser también bastante típico. O bien tras pedir argumentos de entrada o bien pidiendo los datos que hacen falta.

Plantilla completa (la más repetida)

```
#!/bin/bash

mostrar_menu() {
    echo "=== Menú ==="
    echo "1) Opción A"
    echo "2) Opción B"
    echo "3) Salir"
}

# Bucle principal
while true; do
    mostrar_menu
    read -rp "Elige opción: " opt
    case "$opt" in
        1) funcion_a ;;
        2) funcion_b ;;
        3) echo "Adiós"; exit 0 ;;
        *) echo "Opción no válida" ;;
    esac
    echo # Línea en blanco de separación
done
```

[!TIP] Variantes vistas:

- `while (( OPCION != 3 ))` → controlar salida con variable
- `cat <<'MENU' ... MENU` → heredoc para el menú
- Cada opción del `case` llama a una **función** independiente

Patrón 5: Bucles

Esto aparece en casi todos los ejercicios. No se nos puede escapar. Para mí, la opción más fácil es tirar del doble "`(( ))`" ya que nos deja

5a. **for** estilo C (el más versátil)

```
for (( i=0; i<10; i++ )); do
    echo "$i"
done
```

5b. **for** con rango

```
for i in {1..10}; do
    echo "$i"
done
```

5c. **for** sobre ficheros

```
for file in "$DIR"/*; do
    [[ -f "$file" ]] && echo "$file"
done
```

5d. **for** sobre argumentos

```
for arg in "$@"; do
    echo "Argumento: $arg"
done
```

5e. **while** con condición

```
while (( contador < limite )); do
    # ...
    (( contador++ ))
done
```

5f. `while` infinito

```
while true; do
    # ...
    [[ condicion_salida ]] && break
done
```

5g. `while read` (leer fichero línea a línea)

```
while IFS=: read -r campo1 campo2 campo3 _; do
    echo "$campo1"
done < /etc/passwd
```

5h. `until` (se ejecuta mientras sea FALSO)

```
until (( posicion == objetivo )); do
    (( posicion++ ))
done
```

5i. Bucle inverso

```
for (( i=${#palabra}-1; i>=0; i-- )); do
    letra="${palabra:i:1}"
done
```

Patrón 6: Funciones

Las funciones normalmente nos van a ayudar si el examen es largo y queremos reutilizar el contenido. Si el script es corto a veces podemos obviarlo. También en ciertas ocasiones el enunciado del ejercicio nos obligan a usarlas.

6a. Definición y uso

```
mi_funcion() {  
    local param1="$1"  
    local param2="$2"  
    # ...  
    echo "resultado" # Devolver valor por stdout  
}  
  
resultado=$(mi_funcion "arg1" "arg2")
```

6b. Función de error centralizada

```
salir_con_error() {  
    echo "Error: $1" >&2  
    exit 1  
}
```

6c. Función de ayuda

```
mostrar_ayuda() {  
    echo "Uso: $0 [opciones] <fichero>"  
    echo "  -l, --legible      Indica si es legible"  
    echo "  -h, --help         Muestra esta ayuda"  
}
```

6d. Estructura profesional de un script

```
#!/bin/bash
# --- Constantes ---
FICHERO="/ruta/al/fichero"

# --- Funciones auxiliares ---
validar_argumentos() { ... }
obtener_datos() { ... }
procesar() { ... }

# --- Programa principal ---
validar_argumentos "$@"
obtener_datos
procesar
```

Suele ser habitual que los ejercicios se estructuren así:

1. Validación → 2. Obtención de datos → 3. Procesamiento → 4. Salida

Patrón 7: Procesar `/etc/passwd` y `/etc/shadow`

Suele ser habitual que nos pidan detalles de usuarios, de cuentas, de grupos, etc...

7a. Leer `/etc/passwd`

```
# Formato: usuario:x:uid:gid:gecos:home:shell
while IFS=: read -r usuario _ uid _ gecos home shell; do
    if (( uid >= 1000 )) && (( uid != 65534 )); then
        echo "$usuario - $home - $shell"
    fi
done < /etc/passwd
```

7b. Leer `/etc/shadow` (requiere root)

```
while IFS=: read -r usuario_shadow clave_shadow _; do
    if [[ "$usuario_shadow" = "$usuario_buscado" ]]; then
        echo "$clave_shadow"
    fi
done < /etc/shadow
```

7c. Filtrar usuarios con `grep+cut`

```
grep "/bin/bash" /etc/passwd | cut -d: -f1 | sort > usuarios_bash.txt
```

7d. Guardar credenciales en formato custom

```
printf '%s:%s\n' "$usuario" "$hash" >> fichero_claves
```

7e. Cifrar contraseña con `openssl`

```
cifrada=$(openssl passwd -1 -salt a "$password")
```

Patrón 8: Gestión de Usuarios (requiere root)

En varios ejercicios nos piden que el usuario sea root, que exista, que creamos el usuario con su carpeta home, que lo borremos...

8a. Comprobar si somos root

```
# Forma 1 (recomendada)
if (( EUID != 0 )); then
    echo "Necesitas ser root."
    exit 1
fi

# Forma 2
if ! id | grep -q uid=0; then
    echo "No tienes permisos."
    return
fi
```

8b. Comprobar si un usuario existe

```
if id "$usuario" &>/dev/null; then
    echo "El usuario existe"
fi
```

8c. Crear usuario

```
useradd -m -d "/home/$login" "$login"
# Con más opciones:
useradd -m -c "$nombre" -d "$home" -s "$shell" -p "$clave_cifrada" "$usuario"
```

8d. Borrar usuario

```
userdel -r "$usuario" # -r borra también su home
```

8e. Obtener UID

```
uid=$(getent passwd "$user" | cut -d: -f3)
```

Patrón 9: Manipulación de Cadenas

Cortar cadenas, pasarlas a minúsculas, mayúsculas, etc...

Tabla resumen (la más importante para memorizar)

Operación	Sintaxis	Ejemplo
Longitud	<code>\${#var}</code>	<code>\${#palabra}</code> → 4
Subcadena	<code>\${var:pos:len}</code>	<code>\${nombre:0:3}</code> → 3 letras
A minúsculas	<code>\${var,,}</code>	<code>\${texto,,}</code>
A mayúsculas	<code>\${var^^}</code>	<code>\${texto^^}</code>
Quitar sufijo corto	<code>\${var%patron}</code>	<code>\${file%.txt}</code> → quita <code>.txt</code>
Quitar sufijo largo	<code>\${var%%patron}</code>	<code>\${ip%*. *}</code> → primer octeto
Quitar prefijo corto	<code>\${var#patron}</code>	<code>\${path#*/}</code>
Quitar prefijo largo	<code>\${var##patron}</code>	<code>\${ip##*.*}</code> → último octeto
Primer carácter	<code>\${var:0:1}</code>	
Con <code>tr</code> mayúsc.	<code>echo "\$x" tr 'a-z' 'A-Z'</code>	
Valor por defecto	<code>\${var:-default}</code>	<code>\${1:-.}</code>

Ejemplos prácticos vistos

```
# Quitar extensión
nombre="${archivo%.txt}"

# Quitar barra final de ruta
DIR="${DIR%/}"

# Obtener solo el nombre del directorio
nombre="${DIR##*/}"

# Obtener parte de IP
ip_base="${ip%.*}"      # 192.168.1
ultimo="${ip##*.}"      # 5

# Invertir palabra
for (( i=${#palabra}-1; i>=0; i-- )); do
    invertida+="${palabra:i:1}"
done
```

Patrón 10: Redes e IPs

No sería raro que el ejercicio nos incluyera algún asunto de redes.

10a. Validar formato IPv4

```
if [[ ! "$1" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]]; then
    echo "IP no válida"
    exit 1
fi
```

10b. Separar octetos de IP

```
IFS='.' read -r -a octetos <<< "$ip"
# octetos[0]=192, octetos[1]=168, etc.
```

10c. Clasificar IP por clase

```
octeto=${ip%.*}
if (( octeto < 128 )); then echo "A"
elif (( octeto < 192 )); then echo "B"
elif (( octeto < 224 )); then echo "C"
elif (( octeto < 240 )); then echo "D"
else echo "E"
fi
```

10d. Ping a rango de IPs

```
ip_base="${ip%.*}"
ultimo="${ip##*.}"
for (( i=0; i<cantidad; i++ )); do
    ping -c 1 -W 1 "${ip_base}.${(ultimo+i)}" &>/dev/null \
        && echo "activa" || echo "inactiva"
done
```

10e. Consultas de red

```
dig -x "$ip" +short # DNS inverso
ss -tn state established | grep ":$puerto" # Conexiones activas
ip -s -h link show "$interfaz" # Estadísticas interfaz
```

Patrón 11: Redirecciones y Pipes

Las clásicas redirecciones, que podemos usar para crear archivos, evitar que se muestren los errores por consola,...

Tabla resumen

Operación	Sintaxis
stdout a archivo (sobrescribir)	<code>comando > archivo</code>
stdout a archivo (añadir)	<code>comando >> archivo</code>
stderr a /dev/null	<code>comando 2>/dev/null</code>
Ambos a /dev/null	<code>comando &>/dev/null</code>
Stderr a stdout	<code>comando 2>&1</code>
Pipe (encadenar)	<code>cmd1 cmd2</code>
Vaciar/crear fichero	<code>: > archivo</code>
Agrupar salidas	<code>{ cmd1; cmd2; } >> log</code>
Here string	<code>comando <<< "\$variable"</code>

Ejemplos prácticos

```
# Log de usuarios conectados
{
    echo "Fecha: $(date)"
    who
} >> registro.txt

# Buscar y comprimir
find "$DIR" -name "*.log" -print0 | tar --null -czf logs.tar.gz -T -

# Ordenar palabras de un fichero
tr -s '[:space:]' '\n' < "$FILE" | grep -v '^$' | sort > salida.txt
```

Patrón 12: Comandos del Sistema más Usados

Comando	Uso típico
<code>wc</code>	Contar líneas/palabras/chars
<code>grep</code>	Buscar texto en ficheros
<code>find</code>	Buscar ficheros
<code>cut</code>	Extraer campos
<code>sort</code>	Ordenar
<code>tr</code>	Traducir/eliminar caracteres
<code>awk</code>	Procesar columnas
<code>tar</code>	Empaquetar/comprimir
<code>scp/rsync</code>	Copia remota
<code>ps</code>	Procesos
<code>df</code>	Espacio en disco
<code>who/whoami</code>	Usuarios conectados
<code>date</code>	Fecha y hora
<code>basename/dirname</code>	Extraer nombre/ruta
<code>openssl</code>	Cifrar contraseñas
<code>ping</code>	Comprobar conectividad

Recetas: Cómo Combinar Patrones

Receta A: "Script que recibe un fichero y hace algo"

Patrón 1c (validar arg) + Patrón 3b (verificar existencia)
+ Patrón 12 (comando sobre el fichero)

Receta B: "Script que recorre un directorio"

Patrón 1h (DIR con default) + Patrón 3b (verificar dir)
+ Patrón 5c (for sobre ficheros) + Patrón 3a (test -f)

Receta C: "Menú con gestión de sistema"

Patrón 4 (menú while+case)
+ Patrón 6a (funciones para cada opción)
+ Patrón 8a (comprobar root si necesario)

Receta D: "Gestión de usuarios/contraseñas"

Patrón 7a (leer /etc/passwd) + Patrón 8 (gestión usuarios)
+ Patrón 2b (read -s) + Patrón 7e (cifrar openssl)

Receta E: "Informe/auditoría del sistema"

Patrón 1e (getopts) + Patrón 7a (/etc/passwd)
+ Patrón 12 (ps, find, df) + Patrón 11 (redirigir)

Receta F: "Script de redes"

Patrón 10a (validar IP) + Patrón 10b (separar octetos)
+ Patrón 10c-d (clasificar o ping)

Resumen de comparadores

Para texto `[[ "$a" == "$b" ]]`

NÚMEROS `(( a == b ))`

REGEX `[[ "$a" =~ ^[0-9]+$ ]]`

FICHEROS `[[ -f "$a" ]]`

ARITMÉTICA `resultado=$(( a + b ))`

NO uses `[ ]` (corchete simple) ni `-eq` / `-lt` para números. **USA siempre** `[[ ]]` para textos/ficheros y `(( ))` para números.

Cosas que repasar antes de entregar el ejercicio

Consejos finales antes de que entregues el ejercicio al tribunal

- ¿Empieza con `#!/bin/bash` ?
- ¿Valida los argumentos de entrada (`$#` , `-z` , tipo)?
- ¿Comprueba que ficheros/directorios existan antes de usarlos?
- ¿Usa `[[ ]]` para texto y `(( ))` para números?
- ¿Las variables van entrecomilladas `"$variable"` ?
- ¿Usa `exit 1` (o código apropiado) en caso de error?
- ¿Redirige errores con `2>/dev/null` donde corresponde?
- ¿Comprueba permisos de root si manipula usuarios?
- ¿Tiene funciones si el script es largo?
- ¿El código tiene comentarios explicativos?